CSC 6302 - Computer Vision

Practical 1 Report

Seeing Like a Machine: The Image-Processing Toolbox

Munyeme Sinyani
University of Zambia
Target length: 4-6 pages

1. What this practical is about

What I did in Practical 1 is go through the front half of a vision pipeline on my own machine: load a photo, look at colour spaces, add noise and try filters, sharpen, find edges, then use morphology so I can count objects. The brief says a computer does not see a picture - it sees numbers - so everything here is just changing those numbers on purpose.

My three photos for this practical: a bright outdoor bike scene (`colourful.jpg`), a dark breakfast shot (`lowlight.jpg`), and fashion earrings on a card for counting (`coins.jpg` - same filename the code expects). Files live in `images/`.

2. Task 1 - Colour spaces and colour picker (3 marks)

What I did: I loaded my colourful outdoor photo and converted it to HSV and LAB with OpenCV. Then I showed each channel. After that I built a simple HSV colour picker for the lime-green bike.

What each channel encodes (short): In HSV, H is the colour type (hue), S is how strong / pure the colour is (saturation), V is brightness. In LAB, L is lightness, A goes green-red, B goes blue-yellow. I used HSV for picking because I can say "this hue range" without RGB mixing brightness into every channel. That is why the brief says HSV is easier than RGB for this job.

Figure 1. HSV channels of my colourful photo

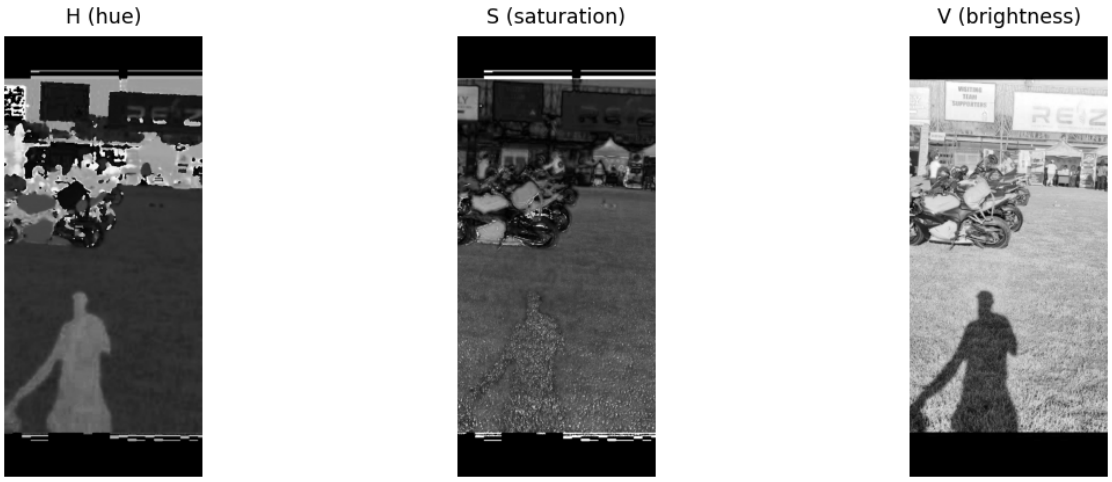


Figure 2. LAB channels

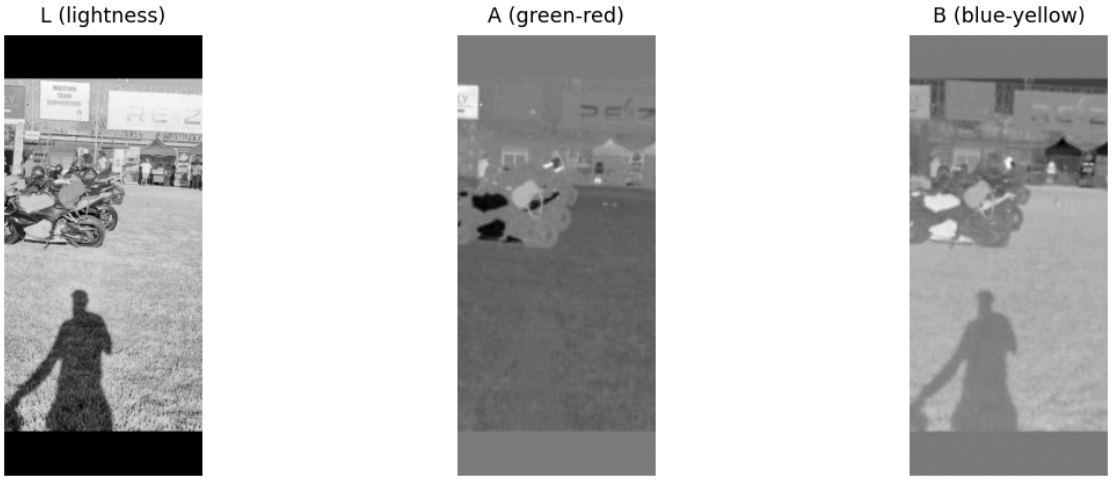
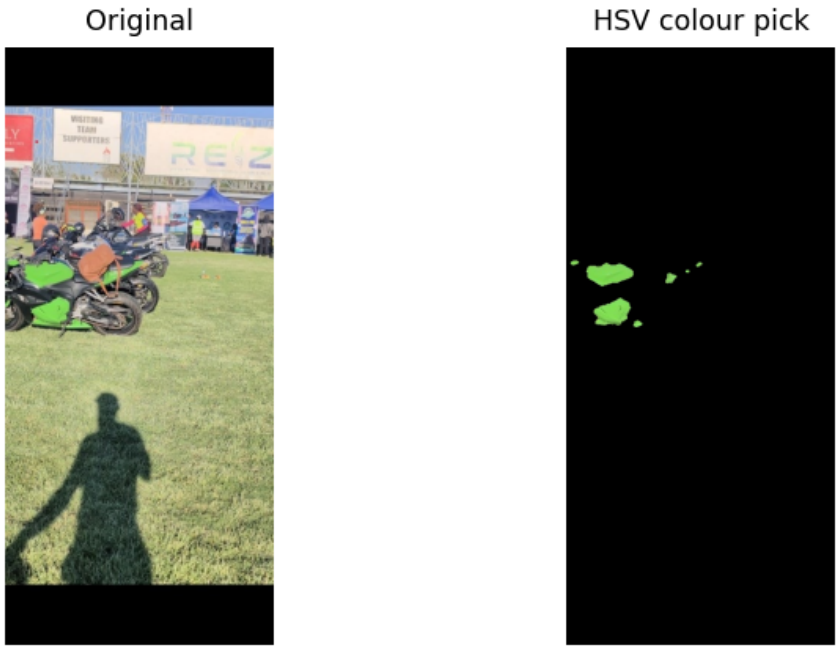


Figure 3. Original vs HSV mask on the lime-green bike



In Figure 3 the mask keeps the bike panels. Grass is a similar hue so I raised the saturation threshold to reduce grass. That is a real limitation I would mention in the viva.

3. Task 2 - Noise and smoothing + my PSNR (4 marks)

What I did: I started from a clean grayscale version of my outdoor photo. I made two corrupted copies - Gaussian noise (sigma = 15) and 5% salt-and-pepper. I restored each with box, Gaussian and median filters at kernel sizes 3, 5 and 7. I implemented PSNR myself in NumPy (no cv2.PSNR).

```
PSNR = 10 * log10(MAX^2 / MSE)
# higher PSNR = restored image closer to the clean one
```

Figure 4. Clean vs Gaussian vs salt-and-pepper



Figure 5. Example restorations

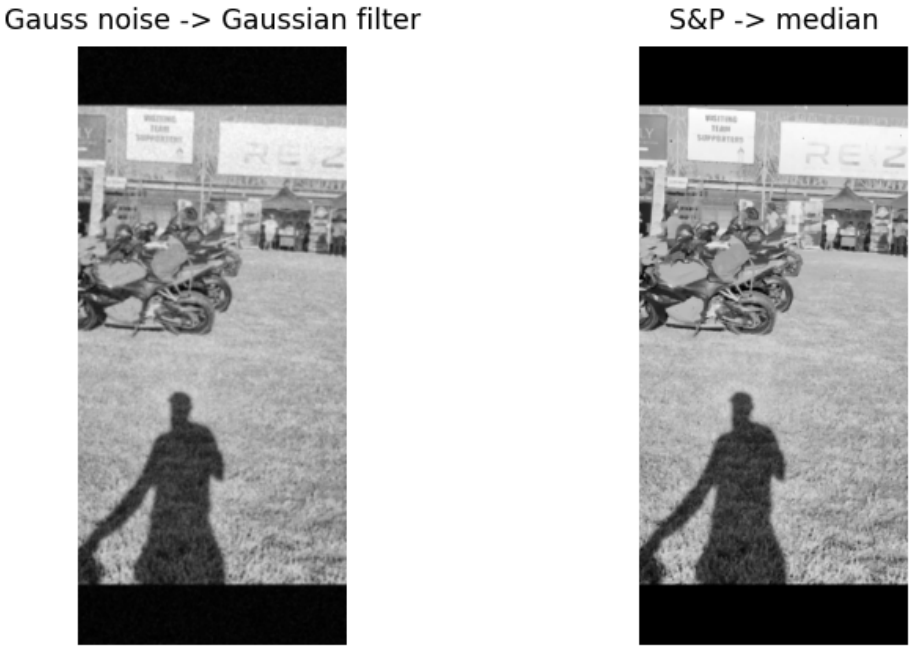


Table 1. PSNR results (dB) from my run

noise	filter	ksize	psnr
gaussian	box	3	23.693
gaussian	gaussian	3	25.149
gaussian	median	3	23.712
gaussian	box	5	22.313
gaussian	gaussian	5	24.008
gaussian	median	5	22.649
gaussian	box	7	21.542
gaussian	gaussian	7	23.057
gaussian	median	7	21.984
sap	box	3	22.191
sap	gaussian	3	22.773
sap	median	3	24.171

sap	box	5	21.78
sap	gaussian	5	22.778
sap	median	5	22.797
sap	box	7	21.246
sap	gaussian	7	22.397
sap	median	7	22.059

Which filter wins and why: For Gaussian noise, the Gaussian filter usually wins (or box is close) because that noise is smooth and spread out - averaging neighbours makes sense. For salt-and-pepper, median wins because outliers are extreme 0/255 spikes; mean/Gaussian smear them, but median throws the spike away. That matches what we said in class about how each filter treats outliers.

4. Task 3 - Sharpening with unsharp masking (2 marks)

What I did: I built sharpened = original + amount * (original - blurred) in float, then clipped to uint8. I tried amount = 0.5, 1.0 and 2.0. At 2.0 I start seeing halo artefacts around strong edges (bright/dark rings). So more amount is not always better.

Figure 6. Unsharp masking amounts



5. Task 4 - Edge detection three ways (4 marks)

What I did: First I wrote my own 2-D convolution and Sobel in NumPy, then compared to cv2.Sobel. After normalising, my max absolute difference was: `max_abs_diff_normalised=0.382839`. Second I ran Canny with three threshold pairs. The two thresholds are hysteresis: strong edges above the high threshold stay, weak edges only stay if they connect to a strong one. Third I ran Laplacian of Gaussian (blur then Laplacian). LoG looks more like thin zero-crossing ridges, while Sobel magnitude is thicker gradient strength.

Figure 7. My Sobel vs OpenCV Sobel

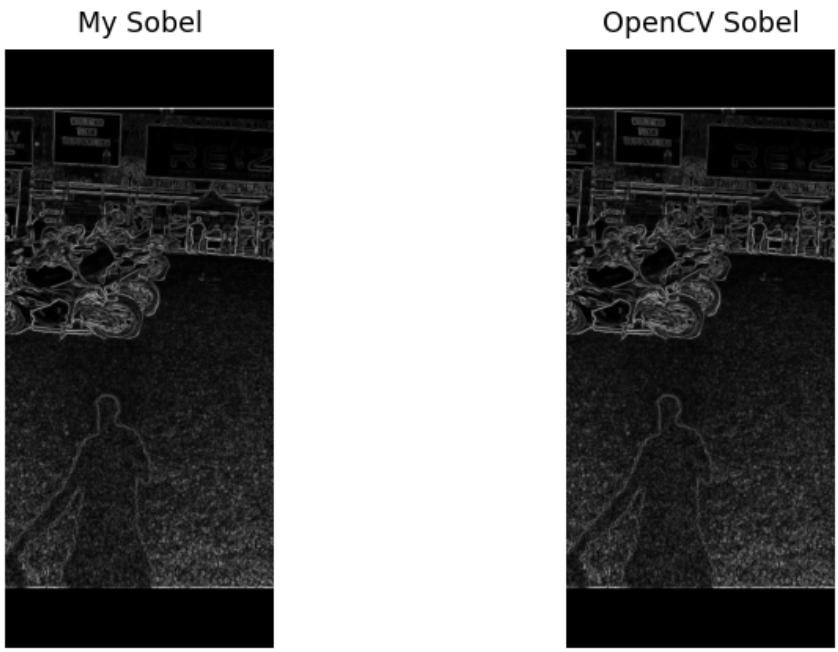


Figure 8. Canny threshold pairs

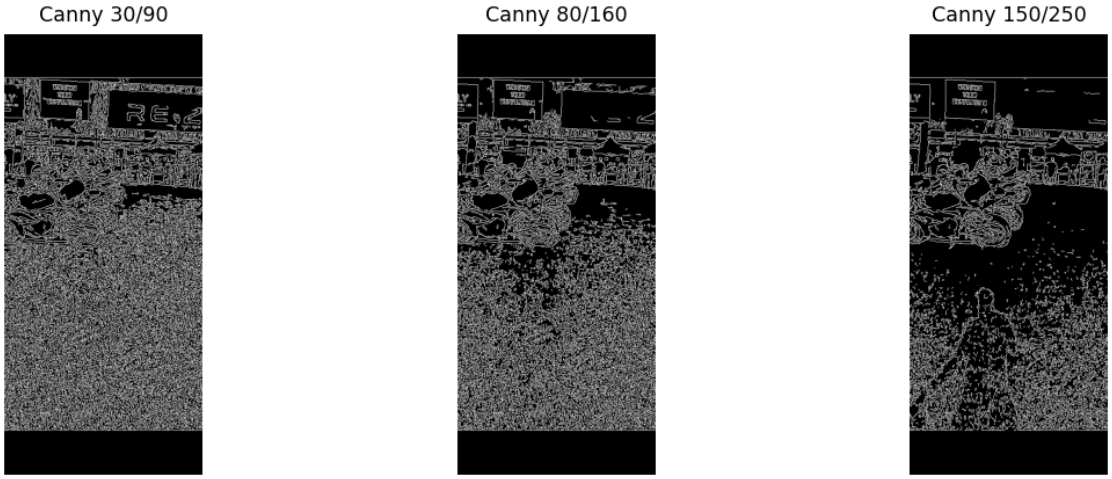
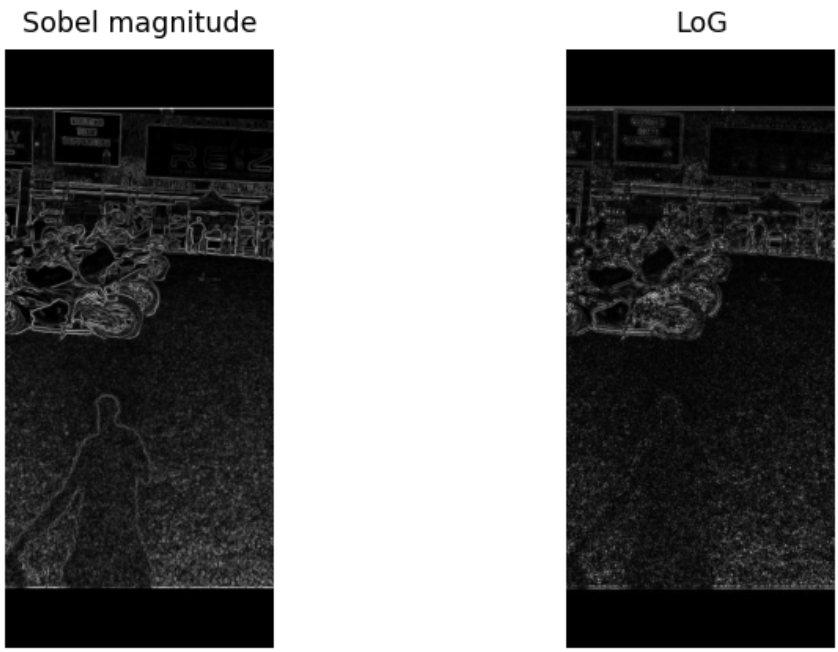


Figure 9. Sobel magnitude vs LoG



6. Task 5 - Morphology and object count (3 marks)

What I did: I used my earrings photo. I thresholded with Otsu (inverted because the card is bright), then erosion, dilation, opening and closing. Structuring element: 3x3 ellipse - small enough that nearby earrings do not glue together too much. I count connected components and also filter by area so tiny text speckles do not count as objects.

Result from my run: `object_count=41 | structuring_element=3x3 ellipse | note=earrings on white card; area filter drops text speckles`. Manual count of earring pieces on the card is lower than a naive blob count because the printed text and card edges also create components. That is a failure mode I talk about in the reflection.

Figure 10. Morphology steps



Figure 11. Count visualisation



7. Task 6 - Pipeline tool + reflection (2 marks)

What I did: I wrapped the ops in `prac1/process.py` so I can run e.g. `python process.py --input images/colourful.jpg --op pick --output out.png`. Supported ops: pick, denoise, sharpen, edges, count.

Reflection (half page)

One place my pipeline fails is object counting when objects sit on a printed card. Connected components happily counts letters in "FASHION EARRINGS" as blobs unless I filter by size. That connects to the lecture idea that thresholding + morphology is not "understanding objects" - it only groups pixels. Lighting also breaks the colour picker: grass and the lime bike share hue, so HSV alone is not enough without a better region prior. Same theme as Assignment 1 - brightness mixes illumination and reflectance, so colour under sun vs shade is not a stable RGB recipe.

8. Reproducibility

Install: `pip install -r requirements.txt`. Generate figures: `python prac1/run_prac1.py`. Code: `process.py`, `run_prac1.py`.

References

- Gonzalez, R. C. & Woods, R. E. Digital Image Processing. (course textbook / lecture notes).
- OpenCV documentation: https://docs.opencv.org/
- Lecture slides for CSC 6302 Computer Vision (UNZA), 2025/26.

AI use

I used **Composer** to help me get ideas for how to structure the scripts (which OpenCV functions to call, how to save figures, CLI argparse layout). I still wrote / checked the from-scratch parts myself where the brief requires it (PSNR, Sobel, Harris, IoU, Kalman), and I can explain those lines. The photos in this report are the ones in my `images/` folder. Any numbers in the tables come from running my code.